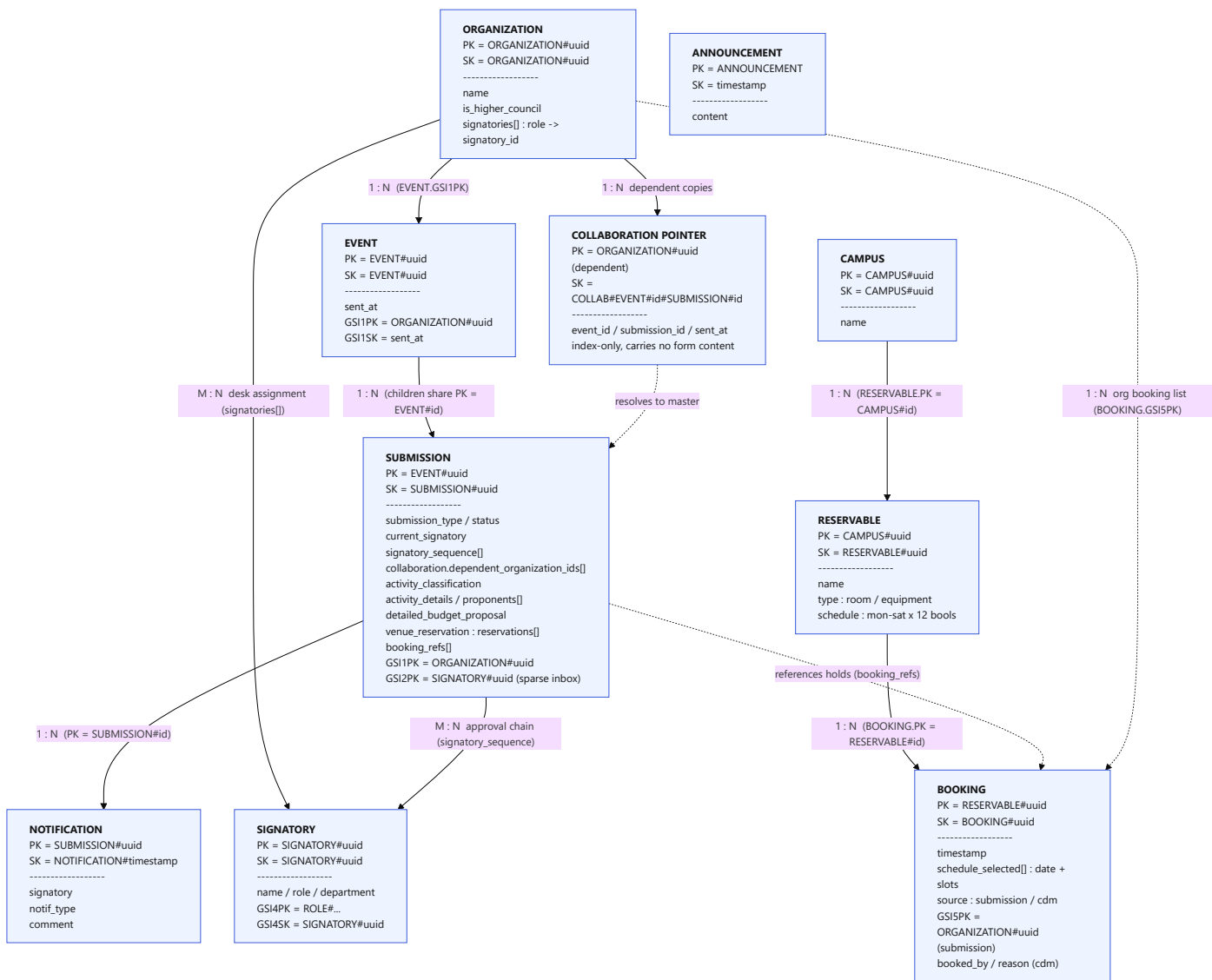


mapua-apex Data Model

DynamoDB single-table data model — conceptual ERD, object dictionary, and access patterns.

Generated 2026-10-11 02:45

Conceptual ERD



Data Objects & Properties

All objects live in a **single DynamoDB table**. Each item is identified by a partition key (`PK`) and a sort key (`SK`); several item types share one partition so related records are read together in a single query. There are **no foreign keys and no cascade**: related IDs are *copied* onto items and resolved at write time, so deleting one item never rewrites another.

ORGANIZATION

The student organization. Also the desk directory: it records which signatory person occupies each approval role for this org.

Property	Type	Notes
<code>PK</code>	string	<code>ORGANIZATION#uuid</code>
<code>SK</code>	string	<code>ORGANIZATION#uuid</code> (same as PK)
<code>name</code>	string	Display name
<code>is_higher_council</code>	boolean	Default <code>false</code> . When true, the Dean step is skipped in routing
<code>signatories</code>	list	Desk directory, one person per role: <code>{ role, signatory_id }</code>
<code>signatories[].role</code>	string	One of <code>adviser</code> / <code>dean</code> / <code>osaar</code> / <code>cdm</code> / <code>admin</code>
<code>signatories[].signatory_id</code>	string	Plain copy of a SIGNATORY id

Allowed desk roles are stored in the order `adviser` → `dean` → `osaar` → `cdm` → `admin` regardless of write order. `admin` may hold a desk but is **not** part of the approval sequence. `osaar` and `cdm` desks are campus-wide (from `OSAAR_SIGNATORY_ID` / `CDM_SIGNATORY_ID` in the API env), not stored per org.

EVENT

A filing/event that owns one or more submissions. Its partition also holds the event's SUBMISSION children.

Property	Type	Notes
<code>PK</code>	string	<code>EVENT#uuid</code>
<code>SK</code>	string	<code>EVENT#uuid</code> (same as PK)
<code>sent_at</code>	timestamp	Creation time (UTC)
<code>GSI1PK</code>	string	<code>ORGANIZATION#uuid</code> — owning org (also identifies the <i>proponent</i> org for collaborations)

Property	Type	Notes
GSI1SK	timestamp	<code>sent_at</code> — for newest-first org event lists

SUBMISSION

A proposal paper (e.g. SAAF) filed under an event. Carries the full form body plus routing state and index keys. This is the single source of truth for a submission, including any collaboration.

Identity, routing & indexes

Property	Type	Notes
PK	string	<code>EVENT#uuid</code> (parent event partition)
SK	string	<code>SUBMISSION#uuid</code>
<code>submission_type</code>	string	e.g. <code>saaf</code>
<code>sent_at</code>	timestamp	Submit time (UTC)
<code>status</code>	string	<code>pending</code> / <code>approved</code> / <code>denied</code> / <code>returned</code>
<code>current_signatory</code>	string	<code>SIGNATORY#uuid</code> holding the paper now
<code>signatory_sequence</code>	list	Ordered <code>SIGNATORY#uuid</code> routing snapshot, written at submit time
<code>collaboration.dependent_organization_ids</code>	list	<code>ORGANIZATION#uuid</code> dependents; empty/absent = no collaboration
GSI1PK	string	<code>ORGANIZATION#uuid</code> (proponent org)
GSI1SK	string	<code>SUBMISSION#uuid</code>
GSI2PK	string	<code>SIGNATORY#uuid</code> — sparse signatory inbox (present while <code>pending</code> / <code>returned</code>)
GSI2SK	timestamp	Desk queue time

activity_classification

Property	Type	Notes
<code>activity_type</code>	string	<code>co-curricular</code> / <code>extra-curricular</code>
<code>total_org_members</code>	number	

Property	Type	Notes
nature	string	major / minor — set by OSAAR via the classification PATCH, not by the student create body

proponents[] (list of people)

id, position_title, first_name, middle_name, last_name, suffix, student_number, program_and_year, date_of_submission, department, position_of_applicant, org_or_course_section, contact_number, email_address, facebook_link.

activity_details

title_and_nature, description, objectives, venue, date_of_event, day_of_event, time_of_event, expected_participants, individual_contribution, proposed_budget. time_of_event is 24-hour "HH:MM – HH:MM"; saved times fall between 7:00 AM and 9:00 PM, end not earlier than start, and start ≠ end.

institutional_alignment

mission_statements { competitive, research, solutions }, core_values_explanation, peo_explanation, sdg_explanation.

detailed_budget_proposal

items[] { item_no, unit, quantity, price_per_unit, total }, grand_total.

venue_reservation & booking_refs

Property	Type	Notes
has_reservation	boolean	Drives whether the CDM desk joins the routing chain
reservations[]	list	One entry per picked reservable: { reservable_id, campus_id, name, type, selections[], remarks }
reservations[].type	string	room equipment
reservations[].selections[]	list	{ date: "YYYY-MM-DD", slots: [int 0–11] } — 0-based indices into the 07:00–21:00 / 70-min day (12 slots)
booking_refs[]	list	Sparse reverse index { pk: "RESERVABLE#uuid", sk: "BOOKING#uuid" }; present only while the paper holds BOOKINGS. Released + removed on deny/return, reconciled on re-edit

Legacy items stored before this shape may still carry `equipment_requested` / `function_rooms` / `audiovisual_equipment` ; reads tolerate both. On submit each selection is validated against the reservable's weekly template and existing BOOKINGS, then written as `source:"submission"` holds — a slot taken in the meantime aborts with **409** and persists nothing.

NOTIFICATION

One row in a submission's approval/timeline history. Stored in a partition keyed by the submission id (a separate partition from the event).

Property	Type	Notes
PK	string	SUBMISSION#uuid
SK	string	NOTIFICATION#timestamp (identity; UTC second precision)
signatory	string	SIGNATORY#uuid author (snapshot)
notif_type	string	approved / fully approved / denied / returned
comment	string	Mandatory when denied or returned

Approve / return / deny insert a NOTIFICATION row as a side effect. A PUT replaces `notif_type` / `comment` in place and does **not** change the SK.

SIGNATORY

A person (the signatory identity). Has **no** `organization_id` ; desk assignment lives on `ORGANIZATION.signatories` .

Property	Type	Notes
PK	string	SIGNATORY#uuid
SK	string	SIGNATORY#uuid (same as PK)
name	string	
role	string	adviser / cdm / dean / osaar / admin
department	string	Deans only (e.g. SOIT)
GSI4PK	string	ROLE#ADVISER / ROLE#CDM / ROLE#DEAN / ROLE#DEAN#{ DEPARTMENT} / ROLE#OSAAR / ROLE#ADMIN
GSI4SK	string	SIGNATORY#uuid

Other items copy the signatory id rather than join back to SIGNATORY:

`ORGANIZATION.signatories[].signatory_id`, `SUBMISSION.current_signatory` / `GSi2PK`, and `NOTIFICATION.signatory`. Renaming keeps the same uuid; deleting a SIGNATORY does not rewrite those copies.

COLLABORATION POINTER

An index-only item, one per dependent organization, that makes the proponent's submission discoverable in a dependent's dashboard. It carries **no** form content — the master SUBMISSION stays the single source of truth.

Property	Type	Notes
PK	string	<code>ORGANIZATION#uuid</code> — the dependent org (not the proponent)
SK	string	<code>COLLAB#EVENT#uuid#SUBMISSION#uuid</code>
<code>event_id</code>	string	<code>EVENT#uuid</code>
<code>submission_id</code>	string	<code>SUBMISSION#uuid</code>
<code>sent_at</code>	timestamp	

Queried from the dependent's partition (`SK` begins `COLLAB#`); no GSI is needed because DynamoDB cannot index the N values of an array. A dependent may read the shared submission and its NOTIFICATION timeline but cannot edit or resubmit — only the proponent may.

ANNOUNCEMENT

A campus-wide announcement. All announcements share one partition; the SK (timestamp) is the identity. Admin Cognito group only.

Property	Type	Notes
PK	string	<code>ANNOUNCEMENT</code> (constant)
SK	string	<code>timestamp</code> — identity (<code>sent_at</code> in the API); newest-first list
<code>content</code>	string	

A PUT replaces `content` in place and does not change the SK.

CAMPUS

A reservable campus (e.g. `Intramuros Campus`, `Makati Campus`), seeded by osaar on the `/campus` page. The SAAF reservation step and the `activity_details.venue` dropdown read the list.

Property	Type	Notes
PK	string	CAMPUS#uuid
SK	string	CAMPUS#uuid (same as PK)
name	string	Display name

List scans PK begins CAMPUS# AND PK = SK . Delete is guarded: **409** if any RESERVABLE still lives under PK = CAMPUS#id . Rename in place (PUT) to keep the same uuid so RESERVABLE children and stored booking campus_id snapshots stay valid.

RESERVABLE

A bookable room or equipment item under a campus. Its SK is globally unique and doubles as the BOOKING partition key.

Property	Type	Notes
PK	string	CAMPUS#uuid (parent campus)
SK	string	RESERVABLE#uuid
name	string	
type	string	room equipment
schedule	object	Recurring weekly template: monday .. saturday , each exactly 12 booleans (one per 07:00–21:00 / 70- min slot; true = available). Sunday is never reservable; an absent day is all- unavailable

Added/edited by cdm on the /reservables page (CSV for name+type, a grid for the schedule). Delete is guarded: **409** if any BOOKING still occupies PK = RESERVABLE#id .

BOOKING

A concrete hold on a reservable's slots for specific dates. One BOOKING per (reservable, submission) or per (reservable, CDM reserve action). Two flavors share the item type, distinguished by source .

Property	Type	Notes
PK	string	RESERVABLE#uuid
SK	string	BOOKING#uuid
timestamp	timestamp	Create time (UTC)

Property	Type	Notes
<code>schedule_selected</code>	list	<code>[{ date: "YYYY-MM-DD", slots: [int 0-11] }]</code> (normalized)
<code>source</code>	string	<code>submission</code> (SAAF-created hold) <code>cdm</code> (manual CDM hold)
<code>campus_id</code>	string	Bare campus uuid (snapshot)
<code>reservable_name</code>	string	Snapshot at write time
<code>reservable_type</code>	string	<code>room</code> <code>equipment</code> (snapshot)

Submission-sourced (`source: "submission"`) adds GSI5 + traceability, and is released on deny/return/re-edit via the SUBMISSION's `booking_refs` :

Property	Type	Notes
<code>GSI5PK</code>	string	<code>ORGANIZATION#uuid</code> — owning org (org booking list)
<code>GSI5SK</code>	timestamp	<code>timestamp</code>
<code>event_id / submission_id / organization_id</code>	string	Bare uuids for traceability and read-only CDM labelling

Manual (`source: "cdm"`) carries **no** GSI5 and **no** submission linkage (so it never surfaces in an org's list); it is released only by CDM delete:

Property	Type	Notes
<code>booked_by</code>	string	<code>SIGNATORY#uuid</code> (the CDM signatory)
<code>reason</code>	string	Optional purpose note (omitted when blank)

Both flavors share one availability engine: a slot is free only when the weekly template allows it AND no BOOKING (either source) occupies that date+slot. Conflict detection runs on the base RESERVABLE partition — never GSI5 — so it works before the (operator-provisioned) organization index exists.

mapua-apex — DynamoDB Access Patterns

DynamoDB is modeled around **access patterns**, not normalized relations: every query is designed first, then the partition key (PK), sort key (SK), and global secondary indexes (GSI) are chosen to serve it. All patterns hit a **single table**. There are no joins — related IDs are copied onto items at write time.

Index summary:

Index	Key	On	Purpose
Base table	PK / SK	all items	Primary access
GSI1	GSI1PK / GSI1SK	EVENT, SUBMISSION	Org-scoped event list & admin submission list
GSI2	GSI2PK / GSI2SK	SUBMISSION	Sparse signatory inbox (pending / returned)
GSI4	GSI4PK / GSI4SK	SIGNATORY	Role directory (ROLE# ...)
GSI5	GSI5PK / GSI5SK	BOOKING	Org-scoped booking list (submission-sourced holds only; operator-provisioned)

Core access patterns

#	Access pattern	Operation	Key condition	Index
1	Org event list (student dashboard)	Query	GSI1PK = ORGANIZATION#id , GSI1SK desc (first 25 EVENT items)	GSI1
2	Event children (its submissions)	Query	PK = EVENT#id , SK begins SUBMISSION#	Base
3	Student's own submissions	Query	GSI1 for org, then each event partition	GSI1 + Base
4	Collaboration submissions (dependent view)	Query	PK = ORGANIZATION# <dep> , SK begins COLLAB# , then resolve each pointer to its master SUBMISSION	Base
5	Single submission detail	GetItem	PK = EVENT#id , SK = SUBMISSION#id	Base

#	Access pattern	Operation	Key condition	Index
6	Submission timeline / audit	Query	PK = SUBMISSION#id , SK begins NOTIFICATION#	Base
7	Signatory inbox (desk queue)	Query	GS12PK = SIGNATORY#id , GS12SK by time (sparse)	GS12
8	Signatory directory by role	Query	GS14PK = ROLE# ... , GS14SK begins SIGNATORY#	GS14
9	Current signatory profile (/signatories/me)	GetItem	PK = SIGNATORY#id , SK = SIGNATORY#id	Base
10	Admin: all submissions	Query	per org GS11PK = ORGANIZATION#id , GS11SK begins SUBMISSION#	GS11
11	Student org record (/students/organization)	GetItem	PK = ORGANIZATION#id , SK = ORGANIZATION#id	Base
12	Org directory (/students/organizations)	Query	org list → { organization_id, name }	Base
13	Announcement list	Query	PK = ANNOUNCEMENT , SK desc (newest first)	Base
14	Announcement show / edit / delete	GetItem / PutItem / DeleteItem	PK = ANNOUNCEMENT , SK = timestamp	Base
15	Write submission (+ collab pointers)	PutItem	PK = EVENT#id , SK = SUBMISSION#id ; maintain one COLLAB pointer per dependent	Base
16	Approve / return / deny	UpdateItem	advance current_signatory / GS12PK from stored signatory_sequence ; write NOTIFICATION	Base

#	Access pattern	Operation	Key condition	Index
17	Admin: delete organization (guarded)	Query → Deleteltem	block (409) if GSI1PK = ORGANIZATION#id has events/submissions or base PK = ORGANIZATION#id , SK begins COLLAB# ; else Deleteltem PK = SK = ORGANIZATION#id	GSI1 + Base
18	Admin: delete signatory (guarded)	Scan/Query → Deleteltem	block (409) if any ORGANIZATION.sig natories desk holds the id or GSI2PK = SIGNATORY#id (open inbox); else Deleteltem PK = SK = SIGNATORY#id	GSI2 + Base
19	Campus list (osaar /campus , SAAF venue dropdown)	Scan	PK begins CAMPUS# AND PK = SK	Base
20	Reservables under a campus	Query	PK = CAMPUS#id , SK begins RESERVABLE#	Base
21	Reservable availability (template + per-date free/booked slots)	Query	PK = RESERVABLE#id , SK begins BOOKING# ; overlay bookings on the weekly template across the window	Base
22	CDM View/Reserve calendar (raw bookings, both sources)	Query	PK = RESERVABLE#id , SK begins BOOKING# (filtered to dates in window)	Base
23	Create booking (submission hold or CDM manual)	Query → PutItem	validate selections vs template + existing bookings (409 on conflict), then PutItem PK = RESERVABLE#id , SK = BOOKING#id	Base

#	Access pattern	Operation	Key condition	Index
24	Release holds on deny / return / re-edit	DeleteItem	delete each <code>booking_refs[] {pk, sk}</code> (submission holds)	Base
25	Delete a manual booking (CDM)	GetItem → DeleteItem	409 if <code>source</code> is <code>submission</code> ; else DeleteItem PK = <code>RESERVABLE#id</code> , SK = <code>BOOKING#id</code>	Base
26	Admin: delete campus / reservable (guarded)	Query → DeleteItem	block (409) if a campus owns any RESERVABLE, or a reservable owns any BOOKING; else DeleteItem	Base

Page / feature → objects accessed

Page / feature	Objects read or written	Patterns
Student dashboard	ORGANIZATION, EVENT, SUBMISSION, ANNOUNCEMENT, COLLAB POINTER	1, 2, 3, 4, 11, 13
Submission tracker / detail	SUBMISSION, NOTIFICATION, SIGNATORY (by copied id)	5, 6
New / edit submission (SAAF)	SUBMISSION, COLLAB POINTER, ORGANIZATION (directory), CAMPUS, RESERVABLE, BOOKING (holds + <code>booking_refs</code>)	12, 15, 19, 20, 21, 23, 24
Signatory desk (inbox)	SUBMISSION, SIGNATORY	7, 9
Signatory review (approve/return/deny/classify)	SUBMISSION, NOTIFICATION, BOOKING (release holds on deny/return)	6, 16, 24
Admin dashboard (submissions)	ORGANIZATION, SUBMISSION	10
Admin → Organizations	ORGANIZATION, SIGNATORY	11, 12, 17
Admin → Signatories	SIGNATORY, ORGANIZATION (desk guard)	8, 18
osaar → Campus	CAMPUS, RESERVABLE (delete guard)	19, 20, 26
cdm → Reservables (Add/View/Reserve)	CAMPUS, RESERVABLE, BOOKING	20, 21, 22, 23, 25, 26
About / Announcements	ANNOUNCEMENT	13, 14